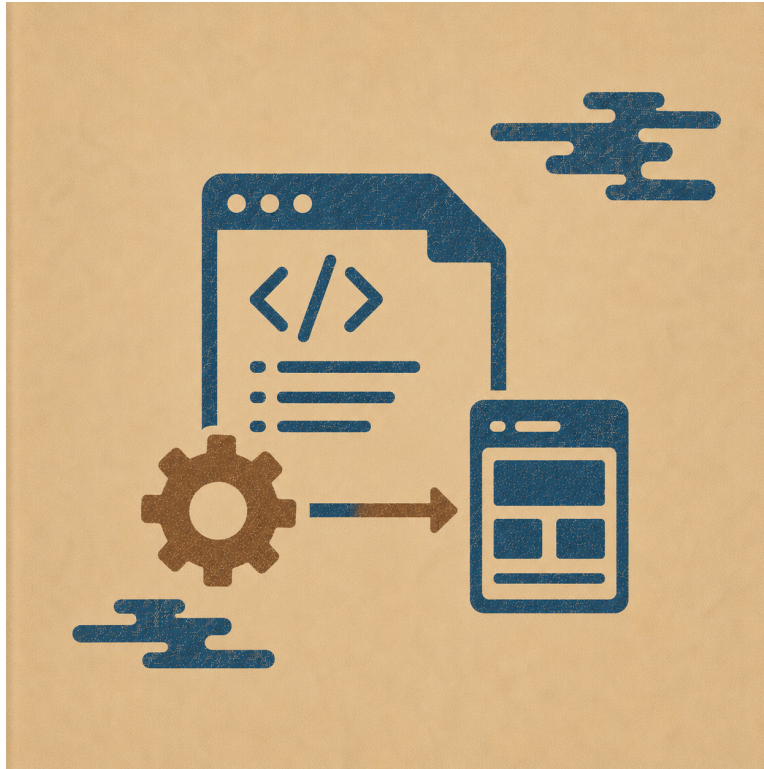


III. SetUp General



Durante estas semanas ya pudimos ver:

- Metodologías
- Estrategias de adopción de Arquitectura
- Servicios disponibles
- Costos de servicios
- Patrones de diseño
- Git

Con esto ya tenemos:

1. Un proyecto
2. Una metodología adecuada para desarrollar los requisitos del mismo
3. Los requisitos
4. Una estrategia de construcción en la nube

5. Seleccionar los servicios y construir la arquitectura completa contemplando el límite de presupuesto
6. Tener la idea general del manejo de las ramas
7. Tener la idea de los patrones de diseño a implementar

¿Ahora que sigue?



Momento de instalación y verificación

Pues es momento de poner manos a la obra y empezar a aprender el desarrollo y construcción del proyecto.

Para esto se necesitará asegurar que contamos con los siguientes elementos instalados en la computadora:

- Git
- Node
- Npm
- Pnpm
- Algún editor de código (VSCode, Cursor, etc...)
- Algún gestor de base de datos (DataGrip, DBeaver, etc...)
- Alguna herramienta para pruebas de apis (Postman, ThunderClient, Etc...)
- Docker Desktop

Verifica cada una de las instalaciones.

Una vez hecho esto generemos nuestro token de git:

Una manera más profesional de clonar repositorios de git, especialmente útil cuando tienes que manejar distintas cuentas en el mismo dispositivo sería de la siguiente manera:

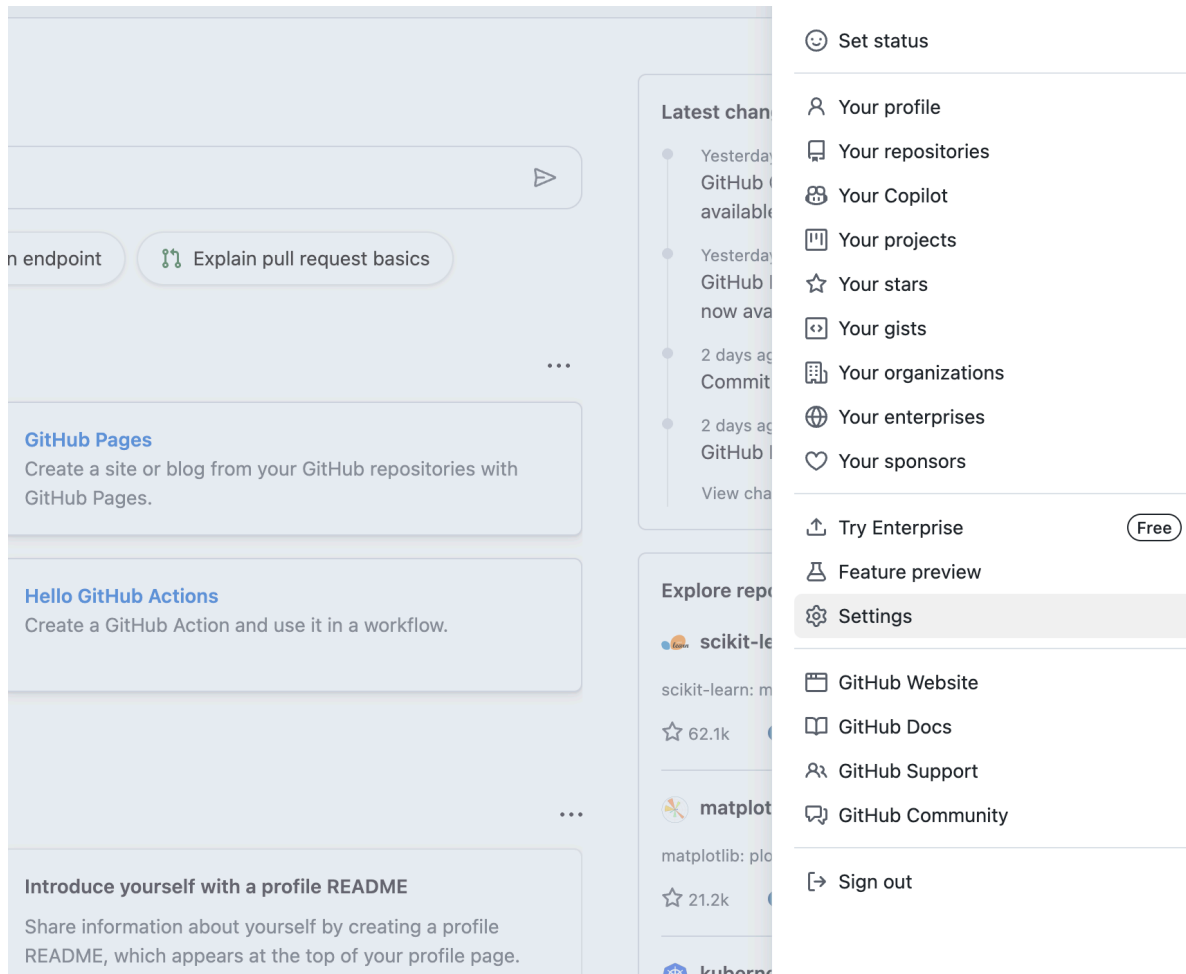
```
git clone https://{{user}}:{{token}}@{{url-git}}
```

Lo que adaptado a un repositorio podría quedar algo así:

```
git clone https://UsuarioPrueba:ghp_aabbkasd@github.com/DASdasD/repositorio.git
```

¿Pero cómo puedo generar ese "token"?

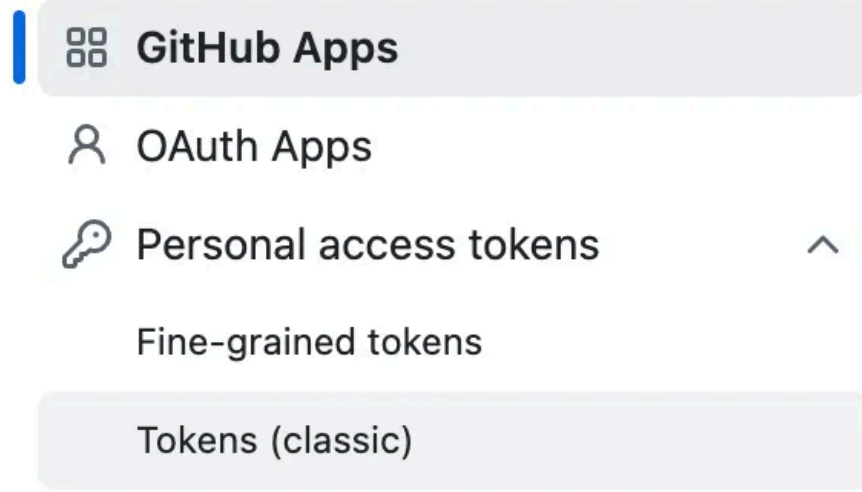
1.- Dentro de Github en las opciones dar click en **Configuración**.



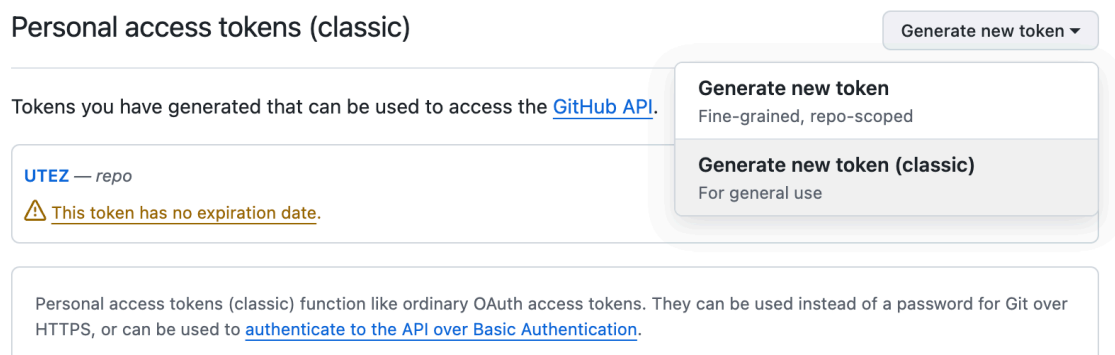
2.-Una vez en la configuración dirígete a **Developer Settings**

- 🌐 Enterprises
 - 💬 Moderation ▼
-
- Code, planning, and automation
- 📁 Repositories
 - 💻 Codespaces
 - 📦 Packages
 - 👤 Copilot
 - 📄 Pages
 - ↩ Saved replies
-
- Security
- 🛡 Code security
-
- Integrations
- 🔌 Applications
 - 🕒 Scheduled reminders
-
- Archives
- 📖 Security log
 - 📖 Sponsorship log
-
- <> Developer settings

3.-Ahora en **Personal Access Tokens** y seleccionas **Tokens (classic)**



4.- Selecciona **Generate new token (classic)**



5.- Puedes ponerle **un nombre, una expiración** y seleccionar **al menos la categoría repo**.

Note

What's this token for?

Expiration

📅 30 days (Jun 16, 2025) ▾

The token will expire on the selected date

Select scopes

Scopes define the access for personal tokens. [Read more about OAuth scopes.](#)

☒ repo	Full control of private repositories
☒ repo:status	Access commit status
☒ repo_deployment	Access deployment status
☒ public_repo	Access public repositories
☒ repo:invite	Access repository invitations
☒ security_events	Read and write security events

Una vez tengas todo listo solo generas el token y listo.



MUY IMPORTANTE:

Cuando salga tu token tienes que copiarlo y guardarlo en un lugar donde nunca lo pierdas ya que no te volverá a salir ese token. Una vez que hayan hecho esto, tendrán que crear un nuevo repo y clonarlo en su equipo para estas prácticas.

Una vez hecho esto creemos los contenedores necesarios que nos servirán más adelante:

Postgres

```
#CAMBIA clase-grupo POR clase-a / clase-b / clase-c / clase-d según sea el caso
docker run --name clase-grupo -e POSTGRES_USER=root -e POSTGRES_PASSWORD=root -e POSTGRES_DB=clase_grupo -p 5432:5432 -d postgres:16
```

Esto te crea un contenedor llamado `clase-grupo` con:

Usuario: `root`

Contraseña: `root`

Base de datos: `clase_grupo`

Puerto: `5432`

Redis

```
#CAMBIA clase-grupo-redis POR clase-a-redis / clase-b-redis
/ clase-c-redis / clase-d-redis según sea el caso
docker run --name clase-grupo-redis -p 6379:6379 -d redis:5
redis-server --requirepass root
```

En Redis normalmente no se usa usuario como en Postgres; aquí el acceso queda con contraseña `root`.

Una vez que ya hicimos esto y tenemos todo clonado y listo...

Recordemos el punto del proyecto.

Categoría	Requisito obligatorio
Arquitectura general	SOFEA
Frontend	Microfrontends como componentes/aplicaciones separadas
Backend	Microservicios
Frontend framework	Next.js con TypeScript
Backend framework	NestJS
Base de datos	PostgreSQL

Entonces ya adentro del proyecto general hagamos este Árbol de carpetas (estructura):

Proyecto/

- |—  Archivos raíz (config del repo)
- |— .agents/ # Skills de agentes (Cursor/Claude)
- |— .claude/ # Config GSD + agentes Claude Code
- |— .husky/ # Git hooks
- |— .planning/ # Planificación GSD (milestones, fases)
- |— agents/ # Gobernanza para IA y desarrollo
- |— apps/
 - | |— addons/ # Herramientas de desarrollo local y extras
 - | |— backend/

gateway/	# API Gateway (punto de entrada HTTP)
commons/	# Libs compartidas backend
services/	# ← microservicios (negocio, no detallado aquí)
test/	# Tests de integración backend
e2e/	# Pruebas end-to-end del monorepo
frontend/	
web-shell/	# App host Next.js (Module Federation)
commons/	# Design system + libs UI compartidas
*-front/	# ← microfrontends (negocio, no detallado aquí)
mobile/	
app-shell/	# App host Flutter
commons/	# Design system + core mobile
plugins/	# ← plugins funcionales (negocio, no detallado aquí)
docs/	# Documentación oficial
infra/	# Infraestructura como código
packages/	# Paquetes compartidos cross-domain
scripts/	# Scripts de automatización del repo
stubs/	# Plantillas/scaffolding para generar apps

Archivos en la raíz

Archivo	Para qué sirve
README.md	Documento principal del proyecto
LICENSE	Licencia del proyecto
package.json	Scripts globales (build , dev , lint , test) y deps raíz
pnpm-workspace.yaml	Declara qué paquetes forman el monorepo
pnpm-lock.yaml	Lockfile de dependencias
turbo.json	Orquestación de tareas entre paquetes (build, dev, lint, test)
tsconfig.base.json	TypeScript compartido (strict, ES2022, source maps)
biome.json	Linter y formateador
commitlint.config.cjs	Reglas de mensajes de commit
bootstrap_scaffolding.sh	Script para inicializar/generar el scaffolding
skills-lock.json	Lock de skills instalados para agentes
.env.example	Variables de entorno de ejemplo
.gitignore / .gitattributes	Config de Git

Archivo	Para qué sirve
<code>.editorconfig</code>	Convenciones de formato entre editores
<code>.nvmrc</code>	Versión de Node recomendada (≥ 20)
<code>.npmrc</code>	Config de pnpm/npm

`.husky/`

Hooks de Git que corren antes de commit/push.

Carpeta/archivo	Para qué sirve
<code>pre-commit</code>	Validaciones antes de cada commit (lint, etc.)
<code>commit-msg</code>	Valida formato del mensaje con commitlint

`docs/`

Documentación oficial del proyecto.

Carpeta	Para qué sirve
<code>00-overview/</code>	Visión, principios y glosario
<code>01-architecture/</code>	Arquitectura formal
<code>01-architecture/c4/</code>	Diagramas C4 (context, container, component, deployment)
<code>01-architecture/decisions/</code>	ADRs (decisiones arquitectónicas)
<code>01-architecture/standards/</code>	Estándares: contratos de módulo, API, seguridad, observabilidad, ramas
<code>02-api/</code>	Contratos de API
<code>02-api/openapi/</code>	Specs OpenAPI (gateway + servicios)
<code>02-api/postman/</code>	Colecciones y environments Postman
<code>03-runbooks/</code>	Guías operativas (dev local, deploy QA, incidentes)
<code>04-product/</code>	Roadmap, Figma, estrategia de tenants

`infra/`

Infraestructura como código y operaciones.

Carpeta	Para qué sirve
<code>docker/compose/</code>	Docker Compose por ambiente (<code>dev</code> , <code>qa</code> , <code>prod</code>)

Carpeta	Para qué sirve
<code>docker/images/</code>	Dockerfiles (gateway, servicios, observabilidad)
<code>docker/scripts/</code>	Scripts auxiliares (<code>up</code> , <code>down</code> , reset DB)
<code>k8s/base/</code>	Manifiestos Kubernetes base
<code>k8s/overlays/</code>	Overlays por ambiente (<code>dev</code> , <code>qa</code> , <code>prod</code>)
<code>k8s/helm/</code>	Charts Helm
<code>ci/jenkins/</code>	Jenkinsfile, pipelines y shared libs
<code>ci/scripts/</code>	Scripts de CI (<code>build</code> , <code>test</code> , <code>scan</code>)
<code>observability/otel/</code>	OpenTelemetry
<code>observability/prometheus/</code>	Métricas
<code>observability/grafana/</code>	Dashboards Grafana
<code>observability/dashboards/</code>	Definiciones de dashboards

agents/

Gobernanza para desarrollo humano y agentes de IA.

Carpeta	Para qué sirve
<code>development-flow.md</code>	Flujo oficial: rama → docs → diagramas → backend → frontend → PR
<code>rules/</code>	Reglas por ámbito (arquitectura, seguridad, API, frontend, backend, mobile)
<code>checklists/</code>	Checklists de PR, release, readiness de módulo, threat model
<code>prompts/cursor/</code>	Prompts para Cursor
<code>prompts/chatgpt/</code>	Prompts para ChatGPT
<code>prompts/sonar/</code>	Prompts para corregir hallazgos Sonar
<code>templates/</code>	Plantillas: ADR, contrato de módulo, OpenAPI, README de módulo

apps/ — capas estructurales (sin negocio)

apps/addons/

Carpeta	Para qué sirve
<code>project-dev-stack/</code>	CLI para levantar el stack local completo (<code>pnpm dev:stack</code>)

apps/backend/

Carpeta	Para qué sirve
<code>gateway/</code>	API Gateway NestJS — enrutamiento, auth, rate limiting
<code>commons/contracts/</code>	DTOs y eventos versionados compartidos entre servicios
<code>commons/observability/</code>	Logger, tracing, métricas
<code>commons/security/</code>	JWKS client, guards, crypto
<code>services/</code>	Microservicios de dominio <i>(no detallados aquí)</i>
<code>test/</code>	Tests de integración a nivel backend

apps/frontend/

Carpeta	Para qué sirve
<code>web-shell/</code>	Host Next.js — navegación, auth, carga de microfrontends
<code>commons/</code>	UI atómica, hooks, state (RTK), clientes API, utils
<code>*-front/</code>	Microfrontends de dominio <i>(no detallados aquí)</i>

apps/mobile/

Carpeta	Para qué sirve
<code>app-shell/</code>	Host Flutter — navegación, auth, permisos, runtime de plugins
<code>commons/</code>	Design system, navegación, auth, observabilidad mobile
<code>plugins/</code>	Plugins funcionales <i>(no detallados aquí)</i>

apps/e2e/

Pruebas end-to-end del monorepo (Jest + escenarios cross-app).

packages/

Paquetes compartidos entre dominios (opcionales y explícitos).

Carpeta	Para qué sirve
<code>shared-schemas/</code>	JSON Schema / fragmentos OpenAPI reutilizables
<code>shared-protos/</code>	Definiciones gRPC (si aplica)
<code>shared-devtools/</code>	Configs compartidas, reglas de lint, generadores

scripts/

Automatización a nivel repo.

Script	Para qué sirve
<code>security-suite.sh</code>	Suite de verificaciones de seguridad
<code>milestone-close.sh</code>	Cierre de milestone GSD
<code>check-uat-frontmatter.cjs</code>	Valida frontmatter de docs UAT
<code>es-mx-copy-scan.sh</code>	Escaneo de copy en español MX
<code>verify-finance-schema.sh</code>	Verificación de schema (dominio específico)

stubs/

Plantillas para generar nuevos módulos/apps.

Carpeta	Para qué sirve
<code>next-ts/</code>	Stub de app Next.js + TypeScript para scaffolding

Carpetas generadas (no versionar / ignorar)

Carpeta	Para qué sirve
<code>node_modules/</code>	Dependencias instaladas
<code>.pnpm-store/</code>	Cache local de pnpm
<code>.turbo/</code>	Cache de Turborepo
<code>.uat-logs/</code>	Logs de sesiones UAT
<code>.git/</code>	Historial de Git

Flujo de datos (referencia rápida)

```
Clients (Web / Mobile)
  ↓
App Shell / MicroFrontends
  ↓
API Gateway
  ↓
```

Microservices
↓
Postgres + Redis
↓
Kafka (Event Bus)

Comandos útiles del monorepo

Comando	Qué hace
<code>pnpm install</code>	Instala deps + genera Prisma en servicios
<code>pnpm dev</code>	Levanta todos los <code>dev</code> vía Turbo
<code>pnpm dev:stack</code>	Levanta stack local completo
<code>pnpm build</code>	Build de todos los paquetes
<code>pnpm lint</code>	Lint en todo el monorepo
<code>pnpm test</code>	Tests en todo el monorepo

¿Porque pnpm y no npm?

Porque **pnpm suele ser más rápido, más estricto y ahorra espacio** que npm, especialmente en proyectos modernos con muchas dependencias.

La diferencia principal:

Punto	npm	pnpm
Instalación	Normalmente más pesada	Más rápida en muchos casos
Espacio en disco	Duplica más paquetes entre proyectos	Usa un almacén global y enlaces
<code>node_modules</code>	Más permisivo	Más estricto
Dependencias fantasma	Puede dejar que importes cosas que no declaraste	Normalmente te obliga a declarar bien
Monorepos	Funciona	Muy bueno con workspaces
Compatibilidad	Máxima compatibilidad	Muy buena, pero puede revelar errores ocultos

La razón más fuerte para usar **pnpm** es que no copia todo desde cero para cada proyecto. Guarda los paquetes en un **store global** y luego los enlaza al

proyecto. Entonces, si ya instalaste `react` antes, otro proyecto puede reutilizar ese paquete del store sin descargarlo/copiarlo completo otra vez.